



UNIWERSYTET VIZJA

**Wydział Technologii Informatycznych**

**Kierunek studiów:** Informatyka

**Forma studiów:** studia stacjonarne

**Aysenur Kiymaz**

Nr albumu: 47013

***Aplikacja internetowa GlowHub: kompleksowy system  
zarządzania usługami i rezerwacjami dla salonu  
kosmetycznego z trzema oddziałami z wykorzystaniem  
Node.js, Express i MongoDB***

Projekt napisany pod kierunkiem

dr inż. Dominiki Lisiak-Felickiej

Warszawa, 2026 r.

## Streszczenie

W niniejszym projekcie przedstawiono projekt oraz implementację aplikacji internetowej GlowHub dla centrum urody i estetyki działającego w trzech oddziałach (Warszawa, Kraków, Wrocław).

W wielu salonach kosmetycznych klienci nie mają łatwego dostępu do szczegółowych informacji na temat usług oraz przebiegu zabiegów. Ponadto zarządzanie rezerwacjami często odbywa się ręcznie lub przy użyciu ograniczonych narzędzi, co prowadzi do błędów i obniżenia efektywności. W celu rozwiązania tych problemów opracowano centralny system webowy.

Aplikacja umożliwia użytkownikom zapoznanie się z siedmioma dostępnymi usługami (Mezoterapia, Pielęgnacja Skóry, Laserowe Usuwanie Owłosienia, Slimtone, Drenaż Limfatyczny, Regionalne Wyszczuplanie, Makijaż Permanentny) wraz ze szczegółowymi opisami i zdjęciami na dedykowanych stronach każdego zabiegu. Użytkownicy mają również dostęp do sześciu artykułów blogowych, sekcji FAQ, galerii zdjęć centrum oraz strony „O nas” przedstawiającej misję i wartości salonu. System zawiera formularz kontaktowy oraz interaktywną mapę Google prezentującą lokalizacje wszystkich trzech oddziałów. Interfejs obsługuje dwa języki polski i angielski z możliwością przełączania w czasie rzeczywistym za pomocą przycisku dostępnego na każdej stronie.

Użytkownicy mogą założyć konto przy użyciu adresu e-mail i hasła lub zalogować się jednym kliknięciem przez konto Google (Firebase OAuth 2.0). Hasła przechowywane są w postaci zaszyfrowanej przy użyciu algorytmu bcrypt. System udostępnia również funkcję resetowania hasła poprzez link wysłany na adres e-mail użytkownika.

Moduł rezerwacji wizyt umożliwia wybór oddziału, usługi, daty oraz godziny wizyty. Dostępne sloty czasowe generowane są dynamicznie na podstawie czasu trwania wybranej usługi w godzinach 09:00–18:00. Zajęte terminy są automatycznie ukrywane, rezerwacja w przeszłych datach jest zablokowana, a system technicznie uniemożliwia podwójną rezerwację tego samego terminu. Użytkownicy mogą przeglądać swoje rezerwacje w panelu Moje wizyty oraz edytować lub anulować zaplanowane wizyty.

Aplikacja została zrealizowana zgodnie z architekturą MVC z wykorzystaniem Node.js i Express.js po stronie serwera, MongoDB z Mongoose jako bazy danych oraz EJS jako silnika szablonów. Sesje użytkowników przechowywane są w bazie danych MongoDB przy użyciu biblioteki connect-mongo. System stanowi praktyczne, przyjazne dla użytkownika i gotowe do wdrożenia rozwiązanie cyfrowe dla centrum urody, łączące zarządzanie rezerwacjami z funkcjami informacyjnymi i komunikacyjnymi.

**Słowa kluczowe:** Aplikacja Webowa, System Rezerwacji Wizyt, Architektura MVC, Node.js, Express.js, MongoDB, EJS, Firebase Authentication, bcrypt, Wielojęzyczność

## **Podziękowania**

Autor pracy składa serdeczne podziękowania promotorowi dr inż. Dominice Lisiak-Felickiej za wsparcie merytoryczne, cenne wskazówki oraz pomoc na każdym etapie realizacji pracy.

# RODZIAŁ I

## **Aplikacja internetowa GlowHub: kompleksowy system zarządzania usługami i rezerwacjami dla salonu kosmetycznego z trzema oddziałami z wykorzystaniem Node.js, Express i MongoDB**

### **1.1 Cel Projektu**

GlowHub to strona internetowa centrum urody i estetyki działającego w Polsce w trzech oddziałach: Warszawie, Krakowie i Wrocławiu. Głównym celem projektu jest prezentacja usług centrum, budowanie zaufania wśród potencjalnych klientów oraz całkowite przeniesienie procesu rezerwacji wizyt do środowiska online bez konieczności kontaktu telefonicznego ani osobistego.

#### **Zarządzanie tożsamością użytkownika**

System oferuje dwa sposoby rejestracji i logowania. Pierwszy to rejestracja przy użyciu adresu e-mail i hasła – hasła są szyfrowane z wykorzystaniem biblioteki bcrypt z zastosowaniem 10 rund soli i przechowywane w bazie danych. W przypadku próby rejestracji przy użyciu już zarejestrowanego adresu e-mail wyświetlany jest komunikat ostrzegawczy. Drugi sposób to logowanie przez konto Google – integracja z Google OAuth została zrealizowana przy użyciu infrastruktury Firebase Authentication. Po zalogowaniu przez Google token tożsamości wygenerowany przez Firebase jest przekazywany do serwera i weryfikowany za pośrednictwem Firebase Admin SDK. Jeśli dany adres e-mail nie istnieje w systemie, automatycznie tworzony jest nowy rekord użytkownika; jeśli istnieje, do konta dodawane jest pole googleId.

Dane sesji zarządzane są przez express-session i przechowywane trwale w MongoDB za pomocą connect-mongo. Dzięki temu sesja użytkownika jest zachowywana nawet po ponownym uruchomieniu serwera. Dostęp do stron wymagających uwierzytelnienia (rezerwacja wizyty, lista rezerwacji) jest chroniony przez middleware loginRequired – niezalogowani użytkownicy są automatycznie przekierowywani na stronę logowania.

#### **Prezentacja usług i treści korporacyjne**

W nawigacji zdefiniowano siedem kategorii usług: Mezoterapia, Pielęgnacja Skóry, Laserowe Usuwanie Owłosienia, Regionalne Wyszczuplanie (Masaż G5), Drenaż Limfatyczny, Makijaż Permanentny oraz Slimtone. Każda usługa posiada dedykowaną stronę szczegółową, na której opisano czym jest dana usługa, w jaki sposób jest wykonywana i jakie przynosi efekty. Opublikowano również sześć artykułów blogowych, za pośrednictwem których prezentowane są treści informacyjne z zakresu urody i estetyki.

W ramach stron korporacyjnych dostępne są: strona O nas, kontakt, galeria oraz FAQ. Na stronie kontaktowej wyświetlane są adresy i dane kontaktowe wszystkich trzech oddziałów centrum. Strony polityki prywatności i regulaminu zostały przygotowane zgodnie z przepisami RODO Unii Europejskiej.

### **System rezerwacji wizyt**

Funkcjonalne jądro projektu stanowi system zarządzania rezerwacjami. Pliki `appointmentController.js` i `slotGenerator.js` współpracują ze sobą, realizując następujące funkcje:

Dynamiczne generowanie slotów czasowych : funkcja `generateSlots` w pliku `utils/slotGenerator.js` automatycznie generuje przedziały czasowe rezerwacji w zakresie 09:00–18:00 na podstawie wartości `duration` (czas trwania) zdefiniowanej w bazie danych dla danej usługi. Przykładowo dla usługi trwającej 30 minut generowane są sloty: 09:00, 09:30, 10:00 itd.

Kontrola dostępności w czasie rzeczywistym : po wybraniu przez użytkownika daty przeglądarka wysyła żądanie pod adres `/slots` w celu sprawdzenia zajętych rezerwacji na dany dzień. System pobiera istniejące rezerwacje z bazy danych, usuwa zajęte sloty z listy i prezentuje użytkownikowi wyłącznie dostępne przedziały czasowe.

Ochrona przed podwójną rezerwacją : zarówno podczas tworzenia, jak i aktualizacji rezerwacji sprawdzane jest, czy kombinacja wybranej usługi, daty i godziny już istnieje w bazie danych. W scenariuszu aktualizacji własny rekord rezerwacji jest wykluczany z zapytania przy użyciu operatora `$ne`. Dzięki temu technicznie niemożliwe jest utworzenie dwóch różnych rezerwacji w tym samym przedziale czasowym.

Blokada dat przeszłych : podczas tworzenia i aktualizacji rezerwacji wybrana data jest porównywana z aktualną datą zresetowaną do początku dnia; rezerwacja w przeszłej dacie jest odrzucana przez system.

Użytkownik może przeglądać swoje rezerwacje na stronie `/appointments`, edytować je na stronie `/appointments/:id/edit` oraz anulować za pośrednictwem odnośnika `/appointments/:id/delete`.

### **Obsługa wielu języków**

System obsługuje dwa języki: angielski i polski. Zarządzanie językiem odbywa się za pośrednictwem `middleware` zdefiniowanego w pliku `app.js`. Przy każdym żądaniu odczytywana jest preferencja językowa zapisana w sesji użytkownika i ładowane są odpowiednie dane językowe z pliku `config/languages.json`. Funkcja `t()` przekazuje te dane do wszystkich szablonów EJS z użyciem notacji kropkowej, np. `t('login.title')`.

Po kliknięciu przycisku EN lub PL w pasku nawigacji wywoływana jest trasa `/toggle-language/:lang`, preferencja jest zapisywana w sesji, a użytkownik przekierowywany jest na bieżącą stronę. Zmiana języka działa natychmiastowo.

## **Architektura oprogramowania**

Projekt został opracowany zgodnie z architekturą MVC (Model-View-Controller). Warstwa Model (models/) zawiera schematy User, Appointment, Branch i Service zdefiniowane przy użyciu Mongoose. Warstwa View (views/) obejmuje wszystkie strony utworzone przy użyciu silnika szablonów EJS; komponenty partials/header.ejs i partials/footer.ejs są współdzielone przez wszystkie strony. Warstwa Controller (controllers/) zarządza logiką biznesową w oddzielnych plikach, niezależnie od tras. Warstwa Route (routes/) zawiera trasy autoryzacji, rezerwacji, usług i stron zdefiniowane w oddzielnych plikach i podłączone do systemu przez app.js. Ta struktura architektoniczna umożliwia dodanie nowych funkcjonalności takich jak panel administracyjny, powiadomienia e-mail i system płatności przy minimalnej ingerencji w istniejący kod.

### **1.2 Opis projektu**

GlowHub jest aplikacją internetową zaprojektowaną dla salonu kosmetycznego działającego w trzech lokalizacjach (Warszawa, Kraków, Wrocław). System umożliwia użytkownikom przeglądanie dostępnych usług, zapoznanie się z ich szczegółowym opisem oraz korzystanie z treści blogowych związanych z oferowanymi zabiegami. Użytkownicy mogą również uzyskać kompleksowe informacje o salonie oraz dokonywać rezerwacji wizyt online.

Aplikacja oferuje takie funkcjonalności jak system logowania użytkownika (w tym logowanie za pomocą konta Google), moduł rezerwacji wizyt, sekcję najczęściej zadawanych pytań (FAQ), blog informacyjny oraz formularz kontaktowy. Formularz kontaktowy składa się z pól: imię, nazwisko, adres e-mail, numer telefonu oraz treść wiadomości, a także zawiera pole wyboru zgody wymagane w ramach zgodności z RODO. Użytkownicy mogą wybierać oddział, usługę, datę i godzinę wizyty, a także przeglądać, edytować i anulować swoje rezerwacje. System został zaprojektowany jako intuicyjna i dostępna platforma, wspierająca dwa języki (polski i angielski), co umożliwia dotarcie do szerszego grona użytkowników.

### **1.3 Analiza konkurencji**

Jako osoba mieszkająca w Polsce, planując wizytę w salonie kosmetycznym, w pierwszej kolejności analizuję stronę internetową danego salonu. Znalezienie miejsca, które wzbudza pełne zaufanie i sprawia wrażenie profesjonalnego, nie jest łatwe, ponieważ jako klient oczekuję dostępu do szczegółowych, jasnych i przejrzystych informacji na temat oferowanych usług oraz przebiegu zabiegów. W ramach analizy konkurencji przeanalizowano stronę internetową salonu kosmetycznego PLACE działającego w Warszawie. Strona ta prezentuje usługi wraz z informacjami o cenach w sposób przejrzysty i zrozumiały. Po wybraniu konkretnej usługi użytkownik uzyskuje dostęp do szczegółowego opisu oraz sekcji najczęściej zadawanych pytań (FAQ), podobnie jak w aplikacji GlowHub.

Ponadto serwis zawiera takie elementy jak prezentacja salonu, opisy usług, blog, galeria oraz możliwość rezerwacji wizyt za pośrednictwem zewnętrznego systemu Booksy.

Jedną z najważniejszych zalet analizowanego rozwiązania jest jego profesjonalny i estetyczny wygląd, który buduje zaufanie użytkowników. Obecność bloga oraz galerii umożliwia użytkownikom lepsze poznanie oferty salonu i pozytywnie wpływa na ich odczucia.

Przejrzysta prezentacja cen pozwala klientom na wcześniejsze zapoznanie się z kosztami usług. Warto również podkreślić, że strona obsługuje wiele języków, co zwiększa jej dostępność dla szerszego grona odbiorców.

Pomimo licznych zalet rozwiązanie to posiada istotne ograniczenia. Strona nie oferuje indywidualnego konta użytkownika ani dedykowanego systemu zarządzania rezerwacjami. Proces rezerwacji odbywa się za pośrednictwem zewnętrznej platformy, co wymaga przekierowania użytkownika na inną stronę i może negatywnie wpływać na doświadczenie użytkownika. Ponadto brak funkcji personalizacji ogranicza możliwość dostosowania usług do indywidualnych potrzeb klienta. Ograniczona zawartość galerii nie pozwala również w pełni zaprezentować jakości oferowanych usług.

W przeciwieństwie do analizowanego rozwiązania aplikacja GlowHub oferuje w pełni zintegrowany system. Obejmuje on własny moduł rezerwacji, system logowania użytkownika oraz interfejs zarządzania rezerwacjami dostępny z poziomu paska nawigacyjnego, dzięki czemu wszystkie operacje mogą być realizowane w jednym miejscu. Użytkownicy mogą w intuicyjny sposób przeglądać usługi, dokonywać rezerwacji oraz przeglądać, edytować i anulować swoje wizyty. GlowHub łączy funkcjonalność systemu rezerwacji z rozbudowaną warstwą informacyjną, interakcją użytkownika oraz nowoczesnym interfejsem. Dzięki temu stanowi bardziej kompleksowe i profesjonalne rozwiązanie, odpowiadające na potrzeby współczesnych klientów salonów kosmetycznych. System ten wykracza poza funkcję standardowej strony prezentacyjnej, pełniąc rolę zarówno cyfrowej wizytówki salonu, jak i zintegrowanego systemu zarządzania rezerwacjami.

## **1.4 Wykaz zastosowanych technologii**

W ramach realizacji projektu GlowHub wykorzystano szereg technologii umożliwiających stworzenie funkcjonalnej aplikacji webowej wspierającej zarządzanie usługami oraz rezerwacjami w salonie kosmetycznym. Poniżej przedstawiono kluczowe technologie wraz z ich zastosowaniem w projekcie.

## **Technologie backendowe**

1. Node.js – środowisko uruchomieniowe JavaScript po stronie serwera, stanowiące podstawę całej aplikacji.
2. Express.js – framework używany do obsługi routingu, middleware oraz budowy struktury aplikacji webowej.
3. MongoDB – nierelacyjna baza danych wykorzystywana do przechowywania danych użytkowników, rezerwacji, usług oraz oddziałów.
4. Mongoose – biblioteka umożliwiająca modelowanie danych oraz komunikację z bazą MongoDB.
5. express-session – mechanizm zarządzania sesjami użytkowników po zalogowaniu.
6. connect-mongo – narzędzie umożliwiające trwałe przechowywanie sesji w bazie danych MongoDB.
7. bcrypt – biblioteka wykorzystywana do bezpiecznego haszowania i weryfikacji haseł użytkowników.
8. Firebase – usługa używana po stronie przeglądarki do obsługi logowania za pomocą konta Google.
9. Firebase Admin – biblioteka serwerowa służąca do weryfikacji tokenów Google po stronie aplikacji backendowej.
10. method-override – biblioteka pozwalająca na obsługę metod PUT i DELETE w formularzach HTML.
11. dotenv – narzędzie służące do zarządzania zmiennymi środowiskowymi aplikacji.

## **Technologie frontendowe**

1. EJS (Embedded JavaScript Templates) – silnik widoków wykorzystywany do generowania dynamicznych stron HTML po stronie serwera.
2. Bootstrap 5 – framework CSS ładowany za pomocą CDN, używany do tworzenia responsywnego interfejsu użytkownika.
3. Font Awesome 6 – biblioteka ikon ładowana za pomocą CDN, wykorzystywana w elementach interfejsu użytkownika.
4. Google Fonts – zewnętrzna usługa czcionek (CDN); w projekcie zastosowano kroje Inter, Playfair Display oraz Lora.

## **Narzędzia developerskie**

1. nodemon – narzędzie automatycznie restartujące serwer po wykryciu zmian w kodzie źródłowym, używane wyłącznie w środowisku deweloperskim.



## Przykładowe fragmenty kodu

```
const hashed = await bcrypt.hash(password, 10)

const user = await User.findOne({ email })

const match = await bcrypt.compare(password, user.password)

if (!match) {
  return res.render("login", {
    error: "Password is incorrect! Please try again.",
    errorType: "danger"
  })
}

req.session.userId = user._id
```

Rysunek 1: Fragment kodu przedstawiający haszowanie oraz weryfikację hasła użytkownika z wykorzystaniem biblioteki bcrypt

```
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  store: MongoStore.create({
    mongoUrl: process.env.MONGO_URI
  })
}))
```

Rysunek 2: Konfiguracja sesji użytkownika z wykorzystaniem express-session oraz przechowywania danych sesji w bazie MongoDB (connect-mongo)

```
exports.googleLogin = async (req, res) => {
  try {
    const { token } = req.body

    if (!token) {
      return res.status(400).json({ error: "Token required" })
    }

    let decodedToken = {}
```

Rysunek 3.1: Weryfikacja tokena uwierzytelniającego przesyłanego z klienta w procesie logowania przez Google (Firebase Authentication)

```

if (!user) {

  user = await User.create({
    name: name || email || "Google User",
    email: email || "user-" + uid + "@google.com",
    picture: picture || null,
    googleId: uid
  })
} else {

```

**Rysunek 3.2: Tworzenie nowego użytkownika w bazie danych na podstawie danych uzyskanych z konta Google**

```

exports.getLogin = (req, res) => {
  res.render("login")
}

exports.postLogin = async (req, res) => {

  try {

    const { email, password } = req.body

    const user = await User.findOne({ email })

    if (!user) {
      return res.render("login", {
        error: "No account found with this email. Please check or sign up.",
        errorType: "warning"
      })
    }
  }
}

```

**Rysunek 3.3. Obsługa logowania użytkownika z wykorzystaniem adresu e-mail i hasła oraz weryfikacja danych w bazie danych**

```

<!-- Google Login Button -->
<button type="button" id="google-login-btn" class="btn-social">
  <i class="fab fa-google"></i> <%= t('login.google_signin') %>
</button>

```

**Rysunek 4: Fragment interfejsu użytkownika przedstawiający przycisk logowania za pomocą konta Google zintegrowany z Firebase Authentication**

```
const mongoose = require("mongoose")

const userSchema = new mongoose.Schema({
  name: String,

  email: {
    type: String,
    unique: true
  },

  password: String,

  picture: {
    type: String,
    default: null
  },

  googleId: {
    type: String,
    default: null
  },
  |
  createdAt: {
    type: Date,
    default: Date.now
  }
})

module.exports = mongoose.model("User", userSchema)
```

**Rysunek 5: Definicja schematu użytkownika w bazie danych MongoDB z wykorzystaniem biblioteki Mongoose, obejmująca podstawowe pola takie jak imię, adres e-mail, hasło, identyfikator Google oraz datę utworzenia konta**

```
exports.generateSlots = (duration) => {

  const slots = []

  let hour = 9
  let minute = 0

  while(hour < 18){

    const h = hour.toString().padStart(2,"0")
    const m = minute.toString().padStart(2,"0")

    slots.push(`${h}:${m}`)

    minute += duration

    if(minute >= 60){
      hour++
      minute = minute % 60
    }
  }

  return slots
}
```

**Rysunek 6: Algorytm generowania dostępnych slotów czasowych na podstawie czasu trwania usługi, działający w przedziale godzinowym 09:00–18:00**

```

const {serviceId,date} = req.query

const service = await Service.findById(serviceId)

const allSlots = generateSlots(service.duration)

const booked = await Appointment.find({
  service:serviceId,
  date
})

const bookedTimes = booked.map(b=>b.time)

const freeSlots = allSlots.filter(
  s => !bookedTimes.includes(s)
)

```

**Rysunek 7: Filtrowanie dostępnych slotów czasowych poprzez porównanie wszystkich wygenerowanych slotów z już zarezerwowanymi terminami w bazie danych**

```

const selectedDate = new Date(date)
if(selectedDate < new Date(new Date().setHours(0,0,0,0))){
  return res.status(404).render("404")
}

const exists = await Appointment.findOne({
  _id: {$ne: req.params.id},
  service: serviceId,
  date: date,
  time: time
})

```

**Rysunek 8: Walidacja daty rezerwacji - blokowanie rezerwacji w przeszłości oraz zapobieganie podwójnej rezerwacji tego samego terminu**

```

app.use((req, res, next) => {

  if (!req.session.lang) {
    req.session.lang = 'pl'
  }

  const lang = req.session.lang
  const lang_data = languages[lang] || languages['pl']

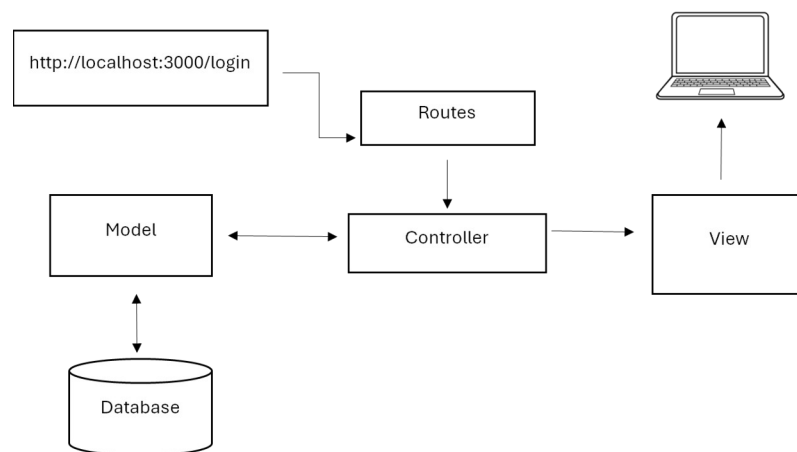
  res.locals.t = (key) => {
    const keys = key.split('.')
    let value = lang_data
    for (let k of keys) {
      value = value[k]
      if (!value) return key
    }
    return value
  }

  res.locals.currentLang = lang
  res.locals.allLanguages = Object.keys(languages)

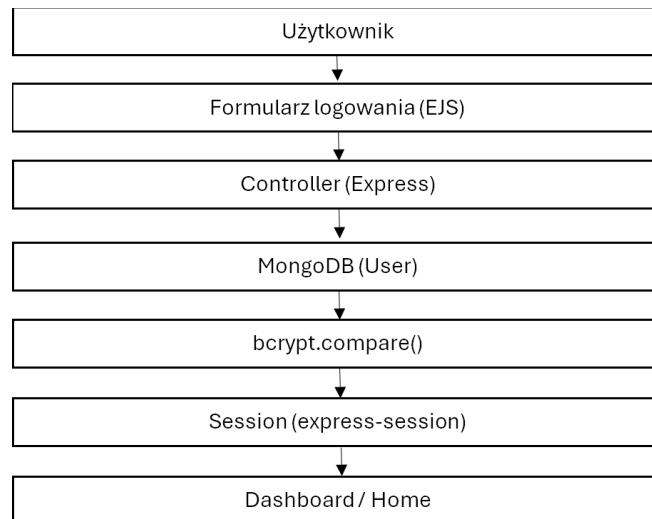
  next()
})

```

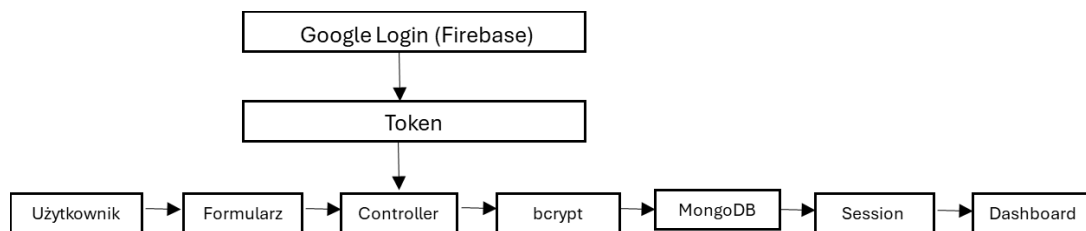
**Rysunek 9: Middleware obsługi wielojęzyczności — automatyczne wykrywanie języka użytkownika z sesji oraz udostępnianie funkcji tłumaczenia t() dla wszystkich widoków EJS**



**Rysunek 10: Diagram architektury systemu przedstawiający przepływ danych w modelu MVC (Model–View–Controller) wraz z komunikacją z bazą danych**



**Rysunek 11: Diagram procesu logowania użytkownika przedstawiający weryfikację danych, komunikację z bazą danych oraz tworzenie sesji użytkownika**



**Rysunek 12: Diagram procesu uwierzytelniania użytkownika uwzględniający logowanie tradycyjne oraz logowanie za pomocą Google (Google Authentication)**

Zastosowane technologie oraz przedstawione fragmenty kodu potwierdzają wykorzystanie nowoczesnych rozwiązań umożliwiających budowę skalowalnej i bezpiecznej aplikacji webowej. Integracja mechanizmów uwierzytelniania, zarządzania sesją oraz obsługi bazy danych pozwoliła na stworzenie kompleksowego systemu spełniającego wymagania współczesnych użytkowników.

## RODZIAŁ II

### Opis stosu technologicznego i uzasadnienie wybranych technologii

#### 2. Kluczowe zagadnienia związane z realizacją projektu

Niniejszy rozdział opisuje kluczowe aspekty implementacyjne aplikacji webowej GlowHub systemu rezerwacji wizyt dla centrum urody i estetyki. Aplikacja została zrealizowana w oparciu o wzorzec architektoniczny MVC (Model-View-Controller) z wykorzystaniem środowiska Node.js oraz frameworka Express.js.

##### 2.1. Architektura aplikacji – Wzorzec MVC

Projekt GlowHub oparty jest na wzorcu MVC (Model-View-Controller), który zapewnia przejrzysty podział odpowiedzialności pomiędzy poszczególnymi warstwami aplikacji. Dzięki temu logika biznesowa, warstwa danych oraz interfejs użytkownika są od siebie niezależne, co ułatwia utrzymanie i rozbudowę systemu.

Struktura katalogów projektu wygląda następująco:

```
glow-hub/
├── app.js           ← punkt wejściowy aplikacji
├── .env             ← zmienne środowiskowe
├── .gitignore
├── package.json
├── seed-branches.js ← inicjalizacja oddziałów
├── seed-services.js ← inicjalizacja usług
├── config/          ← baza danych, Firebase, języki (PL/EN)
├── controllers/     ← logika biznesowa (auth, rezerwacje, usługi)
├── middlewares/     ← loginRequired — ochrona tras
├── models/          ← User, Appointment, Branch, Service
├── routes/          ← trasy HTTP (auth, rezerwacje, usługi)
├── utils/           ← slotGenerator.js
├── public/          ← CSS, obrazy
└── views/          ← 25+ szablonów EJS
```

Warstwa Model odpowiada za definicję schematów bazy danych przy użyciu biblioteki Mongoose oraz operacje na danych przechowywanych w bazie MongoDB. W projekcie zdefiniowano cztery modele: User, Appointment, Branch oraz Service, które odwzorowują strukturę kolekcji w bazie danych i zapewniają walidację oraz typowanie pól.

Warstwa View zawiera szablony EJS (Embedded JavaScript) renderowane po stronie serwera. Szablony mają dostęp do danych przekazywanych przez kontroler oraz do globalnych zmiennych udostępnianych przez middleware, takich jak funkcja tłumaczenia t() oraz dane sesji użytkownika.

Warstwa Controller przetwarza przychodzące żądania HTTP, wywołuje odpowiednie operacje na modelach, a następnie przekazuje dane do widoków lub wykonuje przekierowanie. W projekcie wyróżniono cztery kontrolery: authController (rejestracja i logowanie), appointmentController (zarządzanie rezerwacjami), branchController (oddziały) oraz serviceController (usługi).

## 2.2. Modele danych – baza MongoDB

Dane aplikacji przechowywane są w nierelacyjnej bazie danych MongoDB. Zdefiniowano cztery główne kolekcje: User, Branch, Service oraz Appointment. Poniżej przedstawiono schematy Mongoose dla każdej z nich.

```
const mongoose = require("mongoose")

const userSchema = new mongoose.Schema({
  name: String,
  email: {
    type: String,
    unique: true
  },
  password: String,
  picture: {
    type: String,
    default: null
  },
  googleId: {
    type: String,
    default: null
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
})

module.exports = mongoose.model("User", userSchema)
```

**Rysunek 13: Model użytkownika (User):**

Pole googleId umożliwia powiązanie konta z kontem Google (OAuth). Hasło przechowywane jest w postaci zaszyfrowanej przy użyciu biblioteki bcrypt (algorytm Blowfish, współczynnik kosztu = 10).



```

const mongoose = require("mongoose")

const appointmentSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },

  service: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Service",
    required: true
  },

  branch: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Branch",
    required: true
  },

  date: String,
  time: String,

  status: {
    type: String,
    default: "confirmed"
  }
}, { timestamps: true })

```

**Rysunek 14: Model rezerwacji (Appointment):**

```

const branchSchema = new mongoose.Schema({
  name: String,
  city: String,
  address: String,
  phone: String
})

```

**Rysunek 15: Model oddziału (Branch)**

```

const serviceSchema = new mongoose.Schema({
  name: String,
  price: Number,
  duration: Number,
  branch:{
    type: mongoose.Schema.Types.ObjectId,
    ref:"Branch"
  }
})

```

**Rysunek 16: Model oddziału usługi (Service):**

## 2.3. Rejestracja i logowanie użytkownika

Podczas rejestracji hasło użytkownika jest szyfrowane przy użyciu biblioteki bcrypt z zastosowaniem 10 rund soli, a następnie przechowywane w bazie danych. Próba rejestracji przy użyciu już istniejącego adresu e-mail skutkuje wyświetleniem komunikatu ostrzegawczego.

```
const existing = await User.findOne({ email })

if (existing) {
  return res.render("register", {
    error: "This email address is already in use! Try logging in.",
    errorType: "warning"
  })
}

const hashed = await bcrypt.hash(password, 10)

await User.create({
  name,
  email,
  password: hashed
})
```

**Rysunek 17: Fragment kodu rejestracji użytkownika** sprawdzenie duplikatu adresu e-mail, haszowanie hasła algorytmem bcrypt oraz zapis nowego użytkownika do bazy danych

Podczas logowania pobierany jest rekord użytkownika z bazy danych, a następnie wprowadzone hasło jest porównywane z jego zaszyfrowaną wersją. W przypadku pomyślnej weryfikacji tworzony jest rekord sesji.

```
const match = await bcrypt.compare(password, user.password)

if (!match) {
  return res.render("login", {
    error: "Password is incorrect! Please try again.",
    errorType: "danger"
  })
}

req.session.userId = user._id
req.session.userName = user.name
```

**Rysunek 18: Fragment kodu logowania użytkownika** weryfikacja hasła przy użyciu bcrypt oraz utworzenie rekordu sesji po pomyślnym uwierzytelnieniu

## 2.4. Logowanie przez Google (Firebase Authentication)

Po kliknięciu przycisku logowania przez Google w przeglądarce uruchamiany jest popup Firebase. Wygenerowany token jest przesyłany do serwera i weryfikowany przez Firebase Admin SDK. Jeśli użytkownik nie istnieje w bazie danych, tworzony jest nowy rekord.

```
const provider = new firebase.auth.GoogleAuthProvider();
const result = await auth.signInWithPopup(provider);
const user = result.user;
const token = await user.getIdToken();

const response = await fetch('/google-login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ token })
});
```

**Rysunek 19: Fragment kodu logowania przez Google po stronie przeglądarki – otwarcie popup Firebase, pobranie tokenu tożsamości i przesłanie go do serwera**

```
if (admin.apps.length > 0) {
  try {
    decodedToken = await admin.auth().verifyIdToken(token)
  } catch (err) {
    console.log("Firebase verification disabled - development mode")

    decodedToken = { uid: "dev-" + Date.now(), email: "dev@example.com" }
  }
} else {
  console.log("Firebase Admin not configured - dev mode")

  decodedToken = { uid: "dev-" + Date.now(), email: "dev@example.com" }
}

const { uid, email, name, picture } = decodedToken
```

**Rysunek 20: Fragment kodu logowania przez Google po stronie serwera weryfikacja tokenu Firebase Admin SDK oraz obsługa trybu deweloperskiego**

Powyższy fragment realizuje logowanie przez Google w czterech etapach. W pierwszym kroku Firebase otwiera popup wyboru konta Google i generuje token tożsamości. W drugim kroku token jest przesyłany do serwera. W trzecim kroku serwer weryfikuje token przy użyciu Firebase Admin SDK i na podstawie zdekodowanych danych wyszukuje lub tworzy użytkownika w bazie MongoDB. W ostatnim kroku tworzony jest rekord sesji.

```

let user = await User.findOne({ email })

if (!user) {

  user = await User.create({
    name: name || email || "Google User",
    email: email || "user-" + uid + "@google.com",
    picture: picture || null,
    googleId: uid
  })
} else {

  if (!user.googleId) {
    user.googleId = uid
    await user.save()
  }
}

req.session.userId = user._id
req.session.userName = user.name

```

**Rysunek 21: Fragment kodu logowania przez Google wyszukiwanie lub tworzenie użytkownika oraz utworzenie sesji**

Po zdekodowaniu tokenu serwer wyszukuje użytkownika w bazie danych na podstawie adresu e-mail. Jeśli użytkownik nie istnieje, tworzony jest nowy rekord z danymi pobranymi z konta Google (imię, adres e-mail, zdjęcie profilowe oraz identyfikator Google UID).

W przypadku gdy użytkownik już istnieje, lecz nie posiada przypisanego identyfikatora Google, pole `googleId` zostaje uzupełnione. Na końcu tworzony jest rekord sesji zawierający identyfikator oraz imię użytkownika.

## 2.5. Ochrona stron wymagających uwierzytelnienia

Dostęp do stron wymagających zalogowania jest chroniony przez middleware `loginRequired`. Niezalogowani użytkownicy są automatycznie przekierowywani na stronę logowania.

```

exports.loginRequired = (req, res, next) => {

  if (!req.session || !req.session.userId) {
    return res.redirect("/login")
  }

  next()
}

```

**Rysunek 22: Middleware `loginRequired` ochrona stron wymagających uwierzytelnienia i automatyczne przekierowanie niezalogowanych użytkowników na stronę logowania**

```
router.get(  
  "/appointments",  
  loginRequired,  
  appointmentController.myAppointments  
)
```

**Rysunek 23: Przykład zastosowania middleware loginRequired w trasie ochrona trasy /appointments**

## 2.6. System rezerwacji wizyt

Moduł rezerwacji jest centralną funkcją aplikacji GlowHub. Użytkownik wybiera oddział, usługę, datę oraz dostępny slot czasowy. System sprawdza dostępność slotu w czasie rzeczywistym i zapisuje rezerwację ze statusem confirmed.

Funkcja generateSlots automatycznie generuje dostępne przedziały czasowe w zakresie 09:00–18:00 na podstawie czasu trwania danej usługi zdefiniowanego w bazie danych.

```
exports.generateSlots = (duration) => {  
  const slots = []  
  
  let hour = 9  
  let minute = 0  
  
  while(hour < 18){  
    const h = hour.toString().padStart(2, "0")  
    const m = minute.toString().padStart(2, "0")  
  
    slots.push(`${h}:${m}`)  
  
    minute += duration  
  
    if(minute >= 60){  
      hour++  
      minute = minute % 60  
    }  
  }  
  
  return slots  
}
```

**Rysunek 25: Funkcja generateSlots() – algorytm generowania dostępnych slotów czasowych**

Po wybraniu przez użytkownika daty przeglądarka wysyła zapytanie do endpointu /slots. System pobiera zajęte terminy z bazy danych i zwraca wyłącznie wolne sloty.

```
const allSlots = generateSlots(service.duration)

const booked = await Appointment.find({
  service: serviceId,
  date
})

const bookedTimes = booked.map(b=>b.time)

const freeSlots = allSlots.filter(
  s => !bookedTimes.includes(s)
)
```

**Rysunek 26: Endpoint GET /slots filtrowanie zajętych terminów**

Przed utworzeniem rezerwacji system sprawdza, czy wybrana data nie jest datą przeszłą, a następnie weryfikuje, czy dany slot nie jest już zajęty. Przy aktualizacji rezerwacji własny rekord jest wykluczany z zapytania za pomocą operatora \$ne.

```
if(selectedDate < new Date(today.setHours(0,0,0,0))){
  return res.status(404).render("404")
}
```

**Rysunek 27: Blokada rezerwacji w przeszłych datach**

```
const exists = await Appointment.findOne({
  _id: {$ne: req.params.id},
  service: serviceId,
  date: date,
  time: time
})
```

**Rysunek 28: Fragment kodu ochrony przed podwójną rezerwacją sprawdzenie zajętości slotu podczas tworzenia rezerwacji oraz wykluczenie własnego rekordu operatorem \$ne podczas aktualizacji**

Użytkownik ma również możliwość edycji oraz anulowania istniejącej rezerwacji. Anulowanie polega na trwałym usunięciu rekordu z bazy danych przy użyciu metody findByIdAndDelete. Edycja rezerwacji aktualizuje pola branch, service, date oraz time za pomocą metody findByIdAndUpdate.

```

exports.cancelAppointment = async (req,res)=>{
  try{
    await Appointment.findByIdAndDelete(req.params.id)
    res.redirect("/appointments")
  }catch(err){
    console.log(err)
    return res.status(404).render("404")
  }
}

```

**Rysunek 29: Fragment kodu anulowania rezerwacji usunięcie rekordu z bazy danych i przekierowanie użytkownika**

```

await Appointment.findByIdAndUpdate(req.params.id, {
  branch: branchId,
  service: serviceId,
  date: date,
  time: time
})

```

**Rysunek 30: Fragment kodu aktualizacji rezerwacji nadpisanie pól branch, service, date i time**

## 2.7. System obsługi wielu języków

Aplikacja GlowHub obsługuje dwa języki: polski i angielski. Przy każdym żądaniu middleware odczytuje preferencję językową z sesji użytkownika i udostępnia funkcję tłumaczenia t() wszystkim widokom EJS.

```

app.use((req, res, next) => {

  if (!req.session.lang) {
    req.session.lang = 'pl'
  }

  const lang = req.session.lang
  const lang_data = languages[lang] || languages['pl']

  res.locals.t = (key) => {
    const keys = key.split('.')
    let value = lang_data
    for (let k of keys) {
      value = value[k]
      if (!value) return key
    }
    return value
  }

  res.locals.currentLang = lang
  res.locals.allLanguages = Object.keys(languages)

  next()
})

```

**Rysunek 31: Middleware językowy – funkcja t() dostępna w szablonach EJS**

Zmiana języka odbywa się poprzez wywołanie trasy /toggle-language/:lang, która zapisuje preferencję w sesji i przekierowuje użytkownika z powrotem na bieżącą stronę.

```

app.get('/toggle-language/:lang', (req, res) => {
  const lang = req.params.lang
  if (languages[lang]) {
    req.session.lang = lang
  }
  res.redirect(req.get('referer') || '/')
})

```

**Rysunek 32: Endpoint zmiany języka – przekierowanie na bieżącą stronę**

## 2.8. Zarządzanie sesją i konfiguracja serwera

Dane sesji są zarządzane przez express-session i przechowywane trwale w bazie MongoDB za pomocą connect-mongo. Dzięki temu sesja użytkownika jest zachowywana nawet po ponownym uruchomieniu serwera.



```
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  store: MongoStore.create({
    mongoUrl: process.env.MONGO_URI
  })
}))
```

**Rysunek 33: Konfiguracja sesji z przechowywaniem w MongoDB**

## 2.9. Routing i konfiguracja serwera

Serwer aplikacji uruchamiany jest w pliku app.js. Poniżej przedstawiono kluczowe elementy konfiguracji Express.js: obsługę sesji, statycznych plików oraz rejestrację tras.

```
const app = express()
mongoose.set("bufferCommands", false);
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("Mongo Connected"))
  .catch(err => console.log(err))

app.set("view engine", "ejs")

app.use(express.json())
app.use(express.urlencoded({ extended: true }))
app.use(express.static("public"))
app.use(methodOverride("_method"))
```

**Rysunek 34: Konfiguracja serwera Express.js i połączenie z MongoDB**

```
app.use((req, res, next) => {
  res.locals.session = req.session
  next()
})
```

**Rysunek 35: Middleware globalny udostępnienie sesji wszystkim widokom EJS**

Zmienne środowiskowe (MONGO\_URI, SESSION\_SECRET, klucze Firebase) przechowywane są w pliku .env i wczytywane przez bibliotekę dotenv, co zapewnia bezpieczeństwo danych konfiguracyjnych i oddziela środowisko deweloperskie od produkcyjnego.

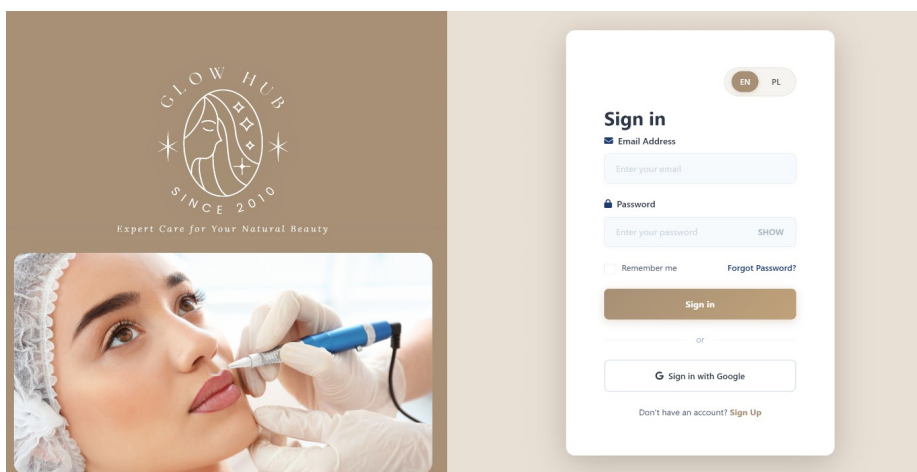
## RODZIAŁ III

### PROJEKT SYSTEMU I FUNKCJONALNOŚCI APLIKACJI

Proces logowania w aplikacji GlowHub został zaprojektowany z myślą o wygodzie i bezpieczeństwie użytkownika. Strona logowania podzielona jest na dwie sekcje. Po lewej stronie znajduje się panel wizualny zawierający logo aplikacji GlowHub wraz z hasłem Expert Care for Your Natural Beauty. Bezpośrednio pod logo znajduje się automatyczny pokaz slajdów prezentujący usługi oferowane przez centrum urody, dzięki czemu użytkownik może zapoznać się z ofertą jeszcze przed zalogowaniem.

Po prawej stronie umieszczony jest formularz logowania. Użytkownik może zalogować się na dwa sposoby. Pierwszym sposobem jest tradycyjne logowanie przy użyciu adresu e-mail i hasła. Formularz zawiera pole tekstowe na adres e-mail, pole hasła z opcją podglądu wpisywanego tekstu (przycisk SHOW) oraz opcję zapamiętania użytkownika (Remember me) wraz z wprowadzonym adresem e-mail dzięki temu przy kolejnej wizycie dane logowania są automatycznie uzupełniane, co znacznie przyspiesza proces logowania. Dostępny jest również link do odzyskiwania hasła (Forgot Password?). Drugim sposobem jest logowanie za pomocą konta Google, zrealizowane przy użyciu Firebase Authentication z integracją konta Google. Po kliknięciu przycisku Sign in with Google użytkownik jest przekierowywany do procesu autoryzacji Google, a jego dane są automatycznie pobierane i zapisywane w bazie danych MongoDB.

W przypadku gdy użytkownik nie posiada jeszcze konta, na dole formularza znajduje się link Sign Up przekierowujący do strony rejestracji. Interfejs dostępny jest w dwóch wersjach językowych polskiej i angielskiej które można przełączać za pomocą przełącznika znajdującego się w prawym górnym rogu ekranu.



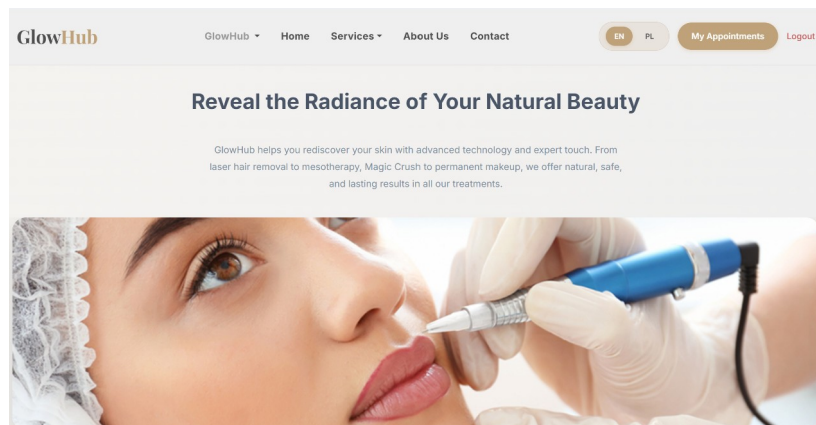
**Rysunek 36: Ekran logowania użytkownika**

Strona rejestracji umożliwia użytkownikom założenie nowego konta w aplikacji GlowHub. Formularz podzielony jest na dwie sekcje.

Po lewej stronie znajduje się panel wizualny z logo aplikacji oraz hasłem zachęcającym do rejestracji. Po prawej stronie umieszczony jest formularz Create Account zawierający pola: imię (First Name), adres e-mail oraz hasło z opcją podglądu (przycisk SHOW). Użytkownik może również zarejestrować się przy użyciu konta Google, klikając przycisk Sign up with Google. W przypadku gdy użytkownik posiada już konto, link Sign in przekierowuje go na stronę logowania. Interfejs dostępny jest w dwóch wersjach językowych polskiej i angielskiej.

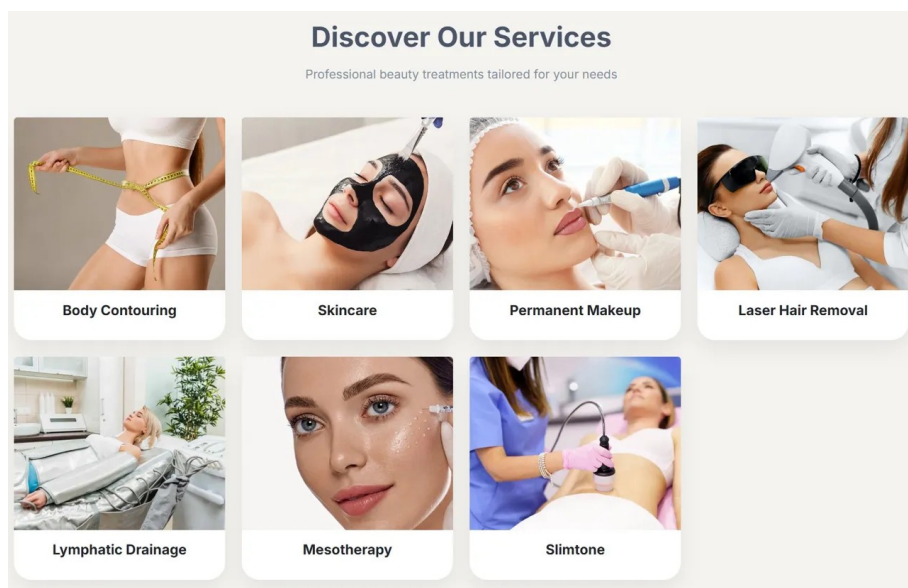
**Rysunek 37: Strona rejestracji użytkownika formularz Create Account z opcją rejestracji przez Google oraz przełącznikiem języka EN/PL**

Poniższe zrzuty ekranu przedstawiają widok strony głównej aplikacji GlowHub uruchomionej w środowisku lokalnym. Strona główna dostępna jest pod adresem głównym aplikacji i stanowi pierwszy punkt kontaktu użytkownika z systemem.



**Rysunek 86: Strona główna aplikacji GlowHub pasek nawigacyjny oraz sekcja powitalna**

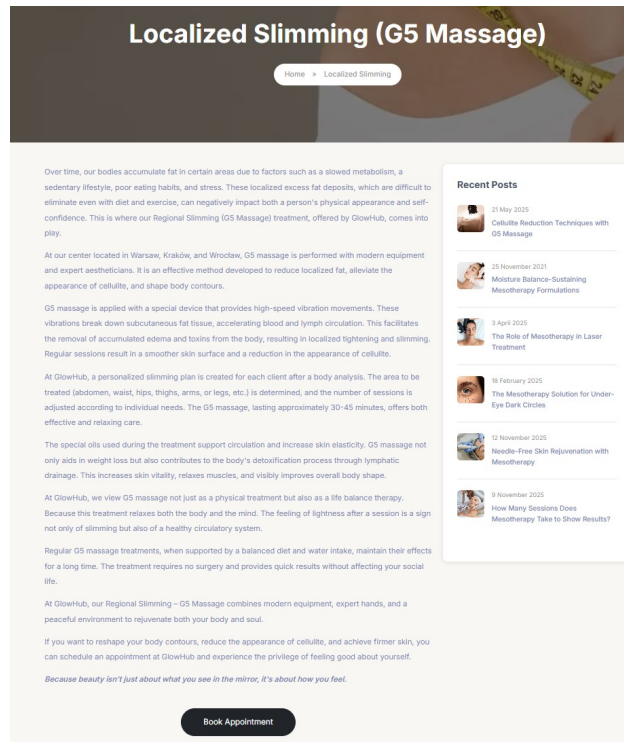
Strona główna zawiera również sekcję Discover Our Services, która prezentuje pełną ofertę zabiegów centrum w formie siatki kart ze zdjęciami. Każda karta zawiera fotografię przedstawiającą dany zabieg oraz jego nazwę.



**Rysunek 38: Strona główna sekcja Discover Our Services z kartami usług**

Po kliknięciu w wybraną usługę użytkownik zostaje przeniesiony na dedykowaną stronę szczegółową danego zabiegu. Strona zawiera szczegółowy opis usługi, jej działanie, efekty oraz zalecenia.

W prawej części ekranu widoczny jest panel Recent Posts prezentujący najnowsze artykuły blogowe związane tematycznie z daną usługą. Panel Recent Posts wyświetla listę powiązanych artykułów blogowych i jest umieszczony statycznie na stronie każdej usługi, umożliwiając użytkownikowi poszerzenie wiedzy na temat wybranych zabiegów. Na dole strony znajduje się przycisk Book Appointment przekierowujący bezpośrednio do systemu rezerwacji.



**Rysunek 39: Strona szczegółowa usługi – opis zabiegu Localized Slimming (G5 Massage) z panelem Recent Posts**

Każda strona usługi zawiera panel Recent Posts po prawej stronie, który prezentuje listę powiązanych artykułów blogowych wraz z miniaturą, datą publikacji oraz tytułem. Po kliknięciu w wybrany artykuł użytkownik zostaje przeniesiony na dedykowaną stronę blogową zawierającą szczegółowy opis tematyczny, zdjęcie główne oraz panel Latest Posts umożliwiający nawigację do innych wpisów. Na dole każdego artykułu znajduje się przycisk Book Appointment przekierowujący użytkownika bezpośrednio do systemu rezerwacji.

### Recent Posts



21 May 2025

Cellulite Reduction Techniques with G5 Massage



25 November 2021

Moisture Balance-Sustaining Mesotherapy Formulations



3 April 2025

The Role of Mesotherapy in Laser Treatment



18 February 2025

The Mesotherapy Solution for Under-Eye Dark Circles



12 November 2025

Needle-Free Skin Rejuvenation with Mesotherapy

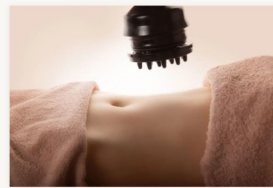


9 November 2025

How Many Sessions Does Mesotherapy Take to Show Results?

## Cellulite Reduction Techniques with G5 Massage

Home » Blog



### Latest Posts

- 21 May 2025  
Cellulite Reduction Techniques with G5 Massage
- 25 November 2021  
Moisture Balance-Sustaining Mesotherapy Formulations
- 3 April 2025  
The Role of Mesotherapy in Laser Treatment
- 18 February 2025  
The Mesotherapy Solution for Under-Eye Dark Circles
- 12 November 2025  
Needle-Free Skin Rejuvenation with Mesotherapy
- 9 November 2025  
How Many Sessions Does Mesotherapy Take to Show Results?

Cellulite is one of the most common skin problems many women complain about, especially during the summer months. Irregular circulation, edema, inactivity, and hormonal effects lead to the orange peel appearance. However, nowadays, reducing the appearance is much easier and more comfortable. GlowHub, located in Warsaw, Kraków, and Wrocław offers G5 massage treatments that reduce cellulite while firming and toning the body.

### What is G5 Massage?

- G5 massage is a professional massage system that works with high-frequency vibration waves.
- This device reaches the deep layers of the body, stimulating muscle tissue, accelerating circulation, and helping to break down fat cells.
- No pain or discomfort is felt during the procedure; on the contrary, it creates a relaxing effect.
- With regular sessions, a significant reduction in the appearance of cellulite and a smoothing of the skin surface are achieved.

### The Effect of G5 Massage on Cellulite Reduction

G5 massage accelerates lymphatic circulation, supporting the elimination of toxins and edema from the body. The breakdown of fat tissue increases skin tightening. This reduces the appearance of cellulite, making the skin smoother and firmer. The treatment yields effective results in the legs, hips, abdomen, and arms.

### Professional G5 Experience at GlowHub

GlowHub creates a personalized G5 program for each client after conducting a body analysis. The treatments are needle-free, contactless, and completely comfortable. If necessary, they are supported with lymphatic drainage or localized draining treatments to achieve maximum results. The individual can immediately return to their daily life after the session; no redness or bruising occurs after the procedure.

G5 massage is a modern and safe method for those who want to reduce the appearance of cellulite and achieve a firmer body. GlowHub offers permanent, needle-free, and painless solutions to cellulite problems with its advanced technology devices and expert team. Discover the GlowHub G5 massage experience for a smoother, firmer, and more aesthetically pleasing body.

Book Appointment

**Rysunek 40: Panel Recent Posts na stronie usługi (po lewej) oraz strona artykułu blogowego z panelem Latest Posts i przyciskiem Book Appointment (po prawej)**

Aplikacja zawiera dedykowaną stronę Frequently Asked Questions, dostępną na dwa sposoby: bezpośrednio ze strony głównej oraz z rozwijanego menu nawigacyjnego GlowHub, które zawiera opcje Galeria i FAQ. Strona umożliwi użytkownikom zapoznanie się z najczęściej zadawanymi pytaniami dotyczącymi zabiegów oferowanych przez centrum. Pytania prezentowane są w formie rozwijanej listy (accordion) po kliknięciu w wybrane pytanie pojawia się szczegółowa odpowiedź, a pozostałe pytania pozostają zwinięte. Dzięki temu interfejs jest przejrzysty i nie przytłacza użytkownika nadmiarem informacji.

## Frequently Asked Questions

Find answers to the most common questions about our beauty services below.

How soon will I see results from mesotherapy?

Mesotherapy results can be visible within 24 hours, though full effects typically appear within 3-7 days. For best results, avoid strenuous activities for 24 hours and apply high-SPF sunscreen.

How do I remove facial hair with laser?

What is the ideal interval between laser sessions?

Is laser epilasyon treatment painful?

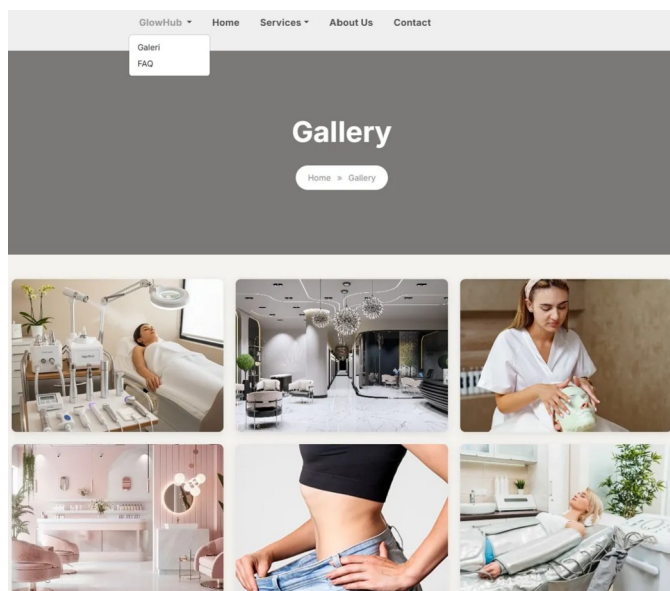
How long does laser epilasyon treatment take?



**Rysunek 41: Strona FAQ z rozwijanymi pytaniami i odpowiedziami dotyczącymi zabiegów**

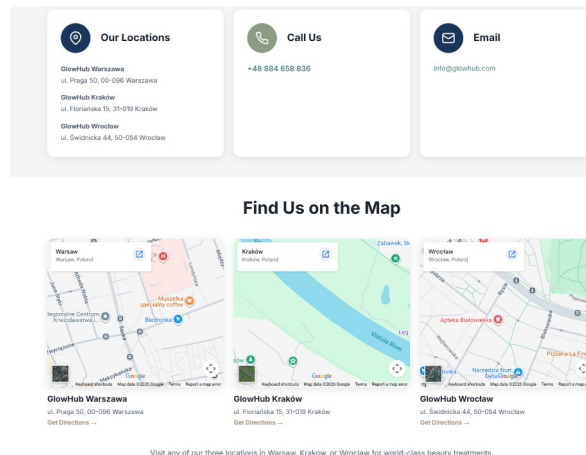


Aplikacja GlowHub zawiera również stronę Galerii, dostępną z menu nawigacyjnego poprzez rozwijane podmenu GlowHub, które zawiera dwie opcje: Galeria (Galeri) oraz FAQ. Strona galerii prezentuje zdjęcia centrum urody, gabinetów zabiegowych oraz wykonywanych zabiegów w formie siatki fotografii. Użytkownik może w ten sposób zapoznać się z atmosferą i standardem centrum przed zarezerwowaniem wizyty.



**Rysunek 42: Strona galerii zdjęcia centrum urody dostępne z menu nawigacyjnego**

Strona kontaktowa aplikacji GlowHub zawiera wszystkie niezbędne informacje umożliwiające skontaktowanie się z centrum. W górnej części strony znajdują się trzy karty informacyjne prezentujące lokalizacje oddziałów, numer telefonu oraz adres e-mail. Poniżej umieszczona jest sekcja Find Us on the Map, która wyświetla interaktywne mapy Google dla każdego z trzech oddziałów centrum w Warszawie, Krakowie i Wrocławiu. Każda mapa zawiera dokładny adres oddziału oraz przycisk Get Directions, który po kliknięciu przekierowuje użytkownika bezpośrednio do aplikacji Google Maps z wyznaczoną trasą do wybranej lokalizacji. Integracja z Google Maps została zrealizowana poprzez osadzenie map w postaci iframe, co umożliwia interaktywne przeglądanie okolicy bez opuszczania strony aplikacji.



**Rysunek 43: Strona kontaktowa – dane kontaktowe oraz interaktywne mapy Google z lokalizacjami oddziałów GlowHub**

Formularz kontaktowy Send Us a Message prezentujący interfejs do kontaktu z centrum. Formularz zawiera pola na imię, nazwisko, adres e-mail, numer telefonu, wybór interesującej usługi oraz treść wiadomości, jednak w obecnej wersji aplikacji nie posiada implementacji backendowej stanowi element interfejsu przewidziany do dalszego rozwoju.

 The image shows a contact form titled 'Send Us a Message'. It contains several input fields: 'First Name \*' and 'Last Name \*' (both with placeholder text 'Your first name' and 'Your last name' respectively), 'Email \*' (placeholder 'your@email.com'), 'Phone \*' (placeholder '+48 XXX XXX-XXXX'), 'Service Interest' (a dropdown menu with 'Select a service...' selected), and 'Message \*' (a text area with placeholder 'Write your message here...'). At the bottom, there is a checkbox labeled 'I agree to the Privacy Policy and Terms of Service' and a brown 'Send Message' button.

**Rysunek 44: Strona kontaktowa – formularz kontaktowy Send Us a Message**

Proces rezerwacji wizyty w aplikacji GlowHub przebiega w trzech krokach, co sygnalizuje pasek postępu widoczny w górnej części formularza. W pierwszym kroku użytkownik wybiera oddział centrum (Select Branch) oraz interesującą go usługę (Select Service) z rozwijanych list. Wszystkie pola są obowiązkowe system informuje o tym użytkownika komunikatem widocznym na dole formularza.

Po dokonaniu wyboru użytkownik klika przycisk Continue Booking, aby przejść do kolejnego kroku, w którym wybiera datę i dostępny slot czasowy.



The screenshot shows a mobile app interface for booking an appointment. At the top, there's a header with a star icon and the text 'Book Your Appointment'. Below it, a subtitle reads 'Select your city, branch, and service to get started'. A progress indicator at the top shows three steps: 1 (active), 2, and 3. The main form area has two sections: 'SELECT BRANCH' with a dropdown menu showing '-- Choose Branch --' and 'SELECT SERVICE' with a dropdown menu showing '-- Choose Service --'. Below these is a large orange button labeled '✓ CONTINUE BOOKING'. At the bottom, a light purple box contains a message: 'All fields are required. Select your preferences to proceed to booking.'

**Rysunek 45: Strona rezerwacji wizyty – krok 1: wybór oddziału i usługi**

W drugim kroku procesu rezerwacji użytkownik widzi podsumowanie wybranej usługi i oddziału, a następnie wybiera datę oraz godzinę wizyty. Lista dostępnych godzin ładowana jest dynamicznie po wybraniu daty – system automatycznie pobiera wolne sloty z serwera i wyświetla tylko te terminy, które nie są jeszcze zajęte. Po wybraniu daty i godziny użytkownik klika przycisk Confirm Booking, co powoduje zapisanie rezerwacji w bazie danych ze statusem confirmed.

**Book Your Appointment**  
Select your preferred date and time to book your beauty treatment

**Complete Your Booking**

**SERVICE**  
SELECTED SERVICE  
Slimtone

**LOCATION**  
BRANCH  
GlowHub Warszawa - ul. Praga 50, 00-096 Warszawa

**SELECT DATE**  
04/28/2026

**SELECT TIME**  
09:30

**CONFIRM BOOKING**

**Rysunek 46: Strona rezerwacji wizyty – krok 2: wybór daty i godziny**

Po pomyślnym zarezerwowaniu wizyty użytkownik zostaje przekierowany na stronę My Appointments, gdzie widoczna jest lista wszystkich jego aktywnych rezerwacji. Każda karta rezerwacji zawiera nazwę oddziału, wybraną usługę, datę oraz godzinę wizyty. Użytkownik ma możliwość edycji rezerwacji poprzez kliknięcie przycisku Edit lub jej usunięcia przyciskiem Delete. Dzięki temu system zapewnia pełną kontrolę nad zaplanowanymi wizytami.

**My Appointments**  
Manage your beauty appointments

GlowHub Warszawa

**SERVICE**  
Slimtone

**TIME**  
09:30

**DATE**  
Tue, Apr 28, 2026

Edit Delete

Back to Home

**Rysunek 47: Strona My Appointments lista aktywnych rezerwacji użytkownika z opcją edycji i usunięcia**

## **RODZIAŁ IV**

### **Wnioski i perspektywy rozwoju**

GlowHub to internetowa platforma rezerwacji wizyt oraz prezentacji informacji dla centrum urody. Głównym celem projektu było stworzenie nowoczesnej aplikacji umożliwiającej klientom szybki dostęp do szczegółowych informacji o oferowanych usługach, a także wygodną rezerwację wizyt online. Założenie to zostało zrealizowane poprzez połączenie funkcji informacyjnych z praktycznym systemem zarządzania rezerwacjami.

W ramach projektu zaimplementowano wiele funkcjonalności wspierających codzienne korzystanie z aplikacji. Użytkownicy mogą tworzyć konta przy użyciu adresu e mail i hasła lub logować się za pomocą konta Google dzięki integracji z Firebase Authentication. Dane sesji przechowywane są w bazie MongoDB, co pozwala na utrzymanie stabilnego procesu logowania.

Istotnym elementem aplikacji jest system rezerwacji wizyt obejmujący siedem różnych usług kosmetycznych. Użytkownik ma możliwość tworzenia, edytowania oraz anulowania rezerwacji, natomiast mechanizm dynamicznych slotów automatycznie ukrywa zajęte terminy i zapobiega podwójnym rezerwacjom. Dzięki temu proces umawiania wizyty przebiega w sposób przejrzysty i bezpieczny.

Aplikacja została przygotowana w dwóch wersjach językowych, polskiej oraz angielskiej. Wszystkie tłumaczenia obsługiwane są centralnie przy użyciu pliku languages.json, co ułatwia zarządzanie treścią oraz dalszy rozwój projektu.

GlowHub zawiera również rozbudowaną część informacyjną. W aplikacji znajdują się szczegółowe opisy zabiegów wraz ze zdjęciami, sekcja FAQ, artykuły blogowe, galeria zdjęć centrum urody oraz strona About Us prezentująca działalność i misję salonu. Dodatkowo użytkownik może skorzystać ze strony kontaktowej wyposażonej w interaktywną mapę Google przedstawiającą lokalizację wszystkich oddziałów.

W projekcie zaimplementowano także panel użytkownika umożliwiający przegląd aktywnych rezerwacji oraz strony dotyczące polityki prywatności i regulaminu. Najważniejsze funkcje aplikacji działają poprawnie, a cały proces od logowania po finalizację rezerwacji został zrealizowany zgodnie z założeniami projektu.

#### **4.1 Napotkane trudności techniczne**

Podczas realizacji projektu pojawiło się kilka wyzwań technicznych. Jednym z nich była integracja logowania Google z wykorzystaniem Firebase Authentication. Proces uwierzytelniania wymagał poprawnej współpracy pomiędzy warstwą klienta a serwerem.

Token uzyskany po stronie przeglądarki musiał zostać zweryfikowany po stronie serwera przy użyciu Firebase Admin SDK, co było konieczne ze względów bezpieczeństwa.

Dodatkową trudność stanowiła obsługa użytkowników posiadających konto utworzone zarówno tradycyjną metodą, jak i przez Google przy użyciu tego samego adresu e-mail.

Kolejnym wyzwaniem było stworzenie mechanizmu obsługi slotów rezerwacji. System musiał dynamicznie generować dostępne godziny wizyt na podstawie czasu trwania wybranej usługi, a następnie synchronizować je z danymi zapisanymi w bazie danych. Najważniejszym elementem było zabezpieczenie przed jednoczesną rezerwacją tego samego terminu przez dwóch użytkowników. Problem rozwiązano poprzez dodatkową weryfikację dostępności terminu bezpośrednio na serwerze w momencie zatwierdzania formularza.

Trudności pojawiły się również podczas implementacji dwujęzycznego interfejsu użytkownika. Wraz ze wzrostem liczby szablonów coraz bardziej wymagające stawało się utrzymanie spójnej struktury kluczy tłumaczeń w pliku `languages.json`. W przyszłości problem ten mógłby zostać rozwiązany poprzez wykorzystanie bardziej rozbudowanego systemu internacjonalizacji, takiego jak biblioteka `i18next`.

## 4.2 Perspektywy rozwoju

Projekt GlowHub może zostać w przyszłości rozbudowany o kolejne funkcjonalności zwiększające wygodę użytkowników oraz możliwości administracyjne systemu.

Jednym z pierwszych usprawnień mogłoby być wdrożenie systemu powiadomień e mail. Po dokonaniu lub anulowaniu rezerwacji użytkownik otrzymywałby automatyczne wiadomości potwierdzające wykonanie operacji. Funkcjonalność ta mogłaby zostać zaimplementowana przy użyciu biblioteki `Nodemailer` i zintegrowana z istniejącym systemem rezerwacji bez konieczności dużych zmian w strukturze aplikacji.

Kolejnym etapem rozwoju może być stworzenie panelu administracyjnego. Rozwiązanie to umożliwiłoby administratorom zarządzanie użytkownikami, usługami oraz wszystkimi rezerwacjami w systemie. Wprowadzenie ról użytkowników pozwoliłoby na oddzielenie funkcji administracyjnych od standardowego konta klienta.

W przyszłości możliwa jest również integracja płatności online przy użyciu systemu `Stripe`. Dzięki temu użytkownicy mogliby opłacać wizyty bezpośrednio podczas procesu rezerwacji. Integracja płatności zwiększyłaby poziom automatyzacji systemu oraz ograniczyłaby konieczność ręcznej obsługi części procesów przez pracowników salonu.

GlowHub stanowi kompletne rozwiązanie wspierające działalność centrum urody. Projekt łączy funkcje rezerwacyjne z rozbudowaną warstwą informacyjną oraz nowoczesnym interfejsem użytkownika. Realizacja aplikacji umożliwiła praktyczne wykorzystanie wzorca MVC, pracy z bazą danych `MongoDB`, integracji z usługami `Firebase` i `Google Maps` oraz implementacji dynamicznych mechanizmów rezerwacji. Planowane w przyszłości rozszerzenia mogą przekształcić aplikację w bardziej kompleksowy system wspierający zarządzanie salonem urody.

## **RODZIAŁ V**

### **Bibliografia/źródła**

#### **Dokumentacja techniczna**

Node.js — oficjalna dokumentacja środowiska uruchomieniowego JavaScript. Dostępna pod adresem: <https://nodejs.org/en/docs>

Express.js — oficjalna dokumentacja frameworka aplikacji webowych. Dostępna pod adresem: <https://expressjs.com>

MongoDB — oficjalna dokumentacja nierelacyjnej bazy danych. Dostępna pod adresem: <https://www.mongodb.com/docs>

Mongoose — oficjalna dokumentacja biblioteki do modelowania danych MongoDB. Dostępna pod adresem: <https://mongoosejs.com/docs>

Firebase Authentication — oficjalna dokumentacja usługi uwierzytelniania Google. Dostępna pod adresem: <https://firebase.google.com/docs/auth>

Bootstrap 5 — oficjalna dokumentacja frameworka CSS. Dostępna pod adresem: <https://getbootstrap.com/docs/5.0>

EJS (Embedded JavaScript Templates) — oficjalna dokumentacja silnika szablonów. Dostępna pod adresem: <https://ejs.co/#docs>

bcrypt — dokumentacja biblioteki do haszowania haseł. Dostępna pod adresem: <https://www.npmjs.com/package/bcrypt>

connect-mongo — dokumentacja biblioteki do przechowywania sesji w MongoDB. Dostępna pod adresem: <https://www.npmjs.com/package/connect-mongo>

express-session — dokumentacja biblioteki do zarządzania sesjami. Dostępna pod adresem: <https://www.npmjs.com/package/express-session>

## Narzędzia deweloperskie

Visual Studio Code — edytor kodu źródłowego. Dostępny pod adresem: <https://code.visualstudio.com>

MongoDB Atlas — chmurowa platforma baz danych. Dostępna pod adresem: <https://www.mongodb.com/atlas>

GitHub — platforma do hostingu i zarządzania kodem źródłowym. Dostępna pod adresem: <https://github.com>

## Zasoby edukacyjne

MDN Web Docs — dokumentacja technologii webowych. Dostępna pod adresem: <https://developer.mozilla.org>

Stack Overflow — społecznościowe forum dla programistów. Dostępne pod adresem: <https://stackoverflow.com>

## Wikipedia

Model-View-Controller — <https://pl.wikipedia.org/wiki/Model-View-Controller>

NoSQL — <https://pl.wikipedia.org/wiki/NoSQL>

Representational State Transfer (REST) — [https://pl.wikipedia.org/wiki/Representational\\_state\\_transfer](https://pl.wikipedia.org/wiki/Representational_state_transfer)

Uwierzytelnianie — <https://pl.wikipedia.org/wiki/Uwierzytelnianie>

HTTP — [https://pl.wikipedia.org/wiki/Hypertext\\_Transfer\\_Protocol](https://pl.wikipedia.org/wiki/Hypertext_Transfer_Protocol)

Sesja (informatyka) — [https://pl.wikipedia.org/wiki/Sesja\\_\(informatyka\)](https://pl.wikipedia.org/wiki/Sesja_(informatyka))

OAuth — <https://pl.wikipedia.org/wiki/OAuth>

Szyfrowanie — <https://pl.wikipedia.org/wiki/Szyfrowanie>